SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I _____Prartana T(192421099)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Prartana T

Annexure A

ANALYTICAL PROBLEM SOLVING

Q1. Apply your understanding of compiler design to demonstrate how a source program is processed through all phases of a compiler.

Consider the statement:

$w=x*y-z+20$

where w,x,y,z are of type real.

Illustrate the transformation of the above statement through each phase of the compiler, including:

(i). Lexical Analysis – Identify tokens and construct the symbol table.

(ii). Syntax Analysis – Show how the statement is validated using grammar rules.

(iii). Semantic Analysis – Verify type consistency and correctness of the expression.

(iv). Intermediate Code Generation – Represent the statement in an intermediate form.

(v). Code Optimization – Apply possible optimizations to improve efficiency.

(vi). Target Code Generation – Generate the final machine-level or assembly-like code.

Clearly show how the internal representation of the program changes at each phase and explain the role of each phase in the translation process. (10 Marks)

Q2. Construct appropriate regular expressions for the following subsets of {0,1}*:

(i) All strings beginning and ending with 1.

(ii) All strings containing the substring 01.

(iii) All strings with exactly one 0.

(iv) All strings with at least two 1's.

(v) All strings that do not end with 0. (10 Marks)

Q3. Apply your understanding of Formal Languages and Automata Theory to interpret regular expressions and construct the corresponding languages.

For each of the following regular expressions, construct and describe the language generated. Represent your answer by listing example strings and explaining the pattern of the language.

(i) $(\epsilon+aa^*)^*$

Construct the language and describe the possible forms of strings generated.

(ii) $(a^*+b^*)^*abb$

Determine the language and explain the structure of strings ending with a specific pattern.

(iii) $(a^*b^*)^*$ aba

Construct the language and identify the mandatory substring condition.

(iii) $(a+ab)*$

Generate the language and describe how repetition affects string formation.

(iv) $(aba)^*+(a^*b^*)^*$

Construct the combined language and explain the contribution of each part of the expression. (10 Marks)

Q4. Apply your understanding of Compiler Design to design components of a lexical analyzer using Lex for a simple programming language.

Identifier Recognition using Lex

You are given a programming language with the following token specifications:

Keywords: if, else, while

Identifiers: A sequence of letters (uppercase and lowercase) and digits, starting with a letter

Constants: Integer values

Operators: +, -, *, /

Delimiters: (,), {, }, ;

(i) Construct a Lex regular expression that recognizes identifiers in this language.

(ii) Justify how your regular expression correctly identifies valid identifiers while avoiding conflicts with keywords.expressions distinguish between valid and invalid numeric formats with suitable examples.

(10 Marks)

Q5. Apply algebraic properties of regular expressions to prove the following identities:

(i) $(0^*1^*)^* = (0+1)^*$

(ii) $(0+1)^*(0+1)^*=(0+1)^*$

(iii) $(0^*)^*=0^*$

(iv) $(0+1)^*0(0+1)^*+(0+1)^*1(0+1)^*=(0+1)^*$

(v) $\varepsilon+0(0)^*=0^*$ (10 Marks)

.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

CO-1 Assessment Tool 1

Analytical Problem solving

(i) Lexical Analysis

converts source code into tokens and remove unnecessary spaces.

<id,1><=><id2><*><id3><->><id4><+><20>

↓

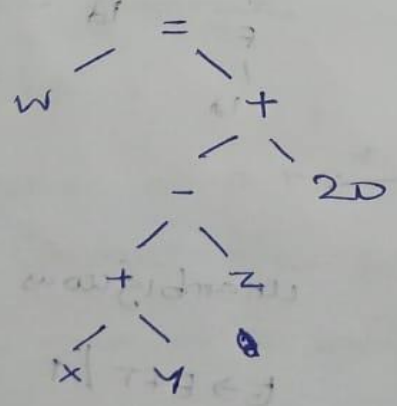
(ii) Syntax Analyzer

checks whether the token sequence follows grammar rules.

S → id = E

E → E*E | E-E | E+E | id | num

Parse structure :-



(iii) Semantic Analysis

x, y, z → Real

20 → Converted to 20.0

Result → Real

Assigned to w (Real)

(iv) Intermediate Code Generation

$t_1 = x * y$

$t_2 = t_1 - z$

$t_3 = \text{int to Real } (20)$

$t_4 = t_2 + t_3$

$w = t_4$

C if constants are directly treated as real

$t_1 = x * y$

$t_2 = t_1 - z$

$t_3 = t_2 + 20.0$

$w = t_3$

(v) Code Optimization

$t_1 = x * y$

$t_1 = t_1 - z$

$w = t_1 + 20.0$

(vi) Target Code generation

```
LOAD  R1 , X
MUL   R1, Y
SUB   R1 , Z
ADD   R1 , #20.0
STORE W, R1
```

2) a (i) All strings beginning & ending with 1

Sol $1(0|1)^*1$

(ii) All strings containing the substring 01

Sol. $(0|1)^*01(0|1)^*$

(iii) All strings with exactly one 0.

1^+01^+

(iv) All strings that do not end with 0

$(0|1)^+1+\varepsilon$

(v) All strings with at least two 1's

$(0|1)^*1(0|1)^*1(0|1)^*$

3)(i) $(\varepsilon + aa)^*$ language :- All strings containing only a's

$\{\varepsilon, a, aa, aaa, aaaa \ldots\}$

(ii) $(a^+ + b^*)^+ abb$ language zero or more occurrences of a only

$\{abb, aabb, babb, aabb, bbabb, ababbb\}$

(iii) $(ab)^+aba$

$\{aba, aaba, baba, ababa, aaababa \ldots\}$

8) (i) $(\epsilon + aa)^*$ Language :- All strings containing only a's

{ ε, a, aa, aaa, aaaa. }

(ii) $(a^* + b^*)^+ a^{bb}$ Language :- 0 or more occurrences of a only.

{ abb, aabb, babb, aaabb, bbabb, ababbb, }

(iii) $(a^b)^+ aba$

{ aba, aaba, baba, ababa, aababa ... }

(iv) $(a + ab)^*$

{ ε, a, ab, aa, aab, aba, abab, aaab.. }

(v) $(aba)^+ + (a^b)^+$

{ ε, aba, abaaba, a, b, aa, bb, aab, abb, abab, baab.. }

4. (i) Lex Regular Expression for identifiers

Lex

[A-Z a-z] [A-Z a-z 0-9]*

Lex Rule

if { printf ("keyword \n"); }

else { printf ("keyword \n"); }

write { printf ("Keyword \n"); }

[A-Z a-z] [A-Z a-z 0-9]*

{printf ("identifier \n");}.

classification

[A-Z a-z]

The First character must be a letter.

[A-Z a-z 0-9]*

The remaining characters can be letter or digits (zero or more).

Eg:

valid identifiers

x

cout

A1

invalid identifiers

1abc (starts with digit)

_num (starts with _)

5) (i) $(0^*|1^*)^+ = (0+1)^+$

$(0+1+) = \{\varepsilon, 0, 1\}$;

$(0+1)^+ = \{\varepsilon, 1, 0, 1, 0, \ldots$

(ii) $(0+1)^+ (0+1)^+ = (0+1)^+$

$(0+)^+(0+1)^+ = \{\varepsilon, 0, \}$

$(0+1)^+ = \{\varepsilon, 0, 1, 11, 00\}$

LHS = RHS

(iii) $(0+)+ = 0^+$

$(r^+)^* = r^+$

LHS = RHS

(iv) $(0+)^+ 0(0+1)^+ + (0+1)+1(0+1)+ = (0+1)+$

$(0+)^+ 0(0+1)^+ + 1(0+1)^+ = (0+1)^+$

(v) $\varepsilon + 0(0)+ 0 = 0+$

$\varepsilon + r(r^*) = r^+$

LHS = RHS